

General advice for your final project report

Synthesized from the grading rubric I published in class and from the feedback I actually left on your progress reports. Treat this as the definitive guide to what I reward and what silently costs points. Everything below maps back to one of the four rubric buckets — **Originality**

Technical soundness **Results** Presentation .

Deliverables & deadline

- **PDF project report (4 pages)**. *Essential*. Use the provided template.
- **Link to source code** — GitHub, Google Drive, or Dropbox. *Required, and the link must actually open for me.*
- **5-minute video walkthrough** (e.g. 5 slides: Motivation → Problem Setup → Model → Experiments → Results). *Not required but encouraged* — useful if I have trouble running your code or parsing the report.

Due: 11:59PM, May 1st. Firm deadline — latest possible.

How you are graded (60 pts)

Originality

15 pts

Novelty in the idea, the application, or the experiments. Am I learning something non-obvious from your report? The clearer the "yes," the closer to full points.

Technical soundness

15 pts

Correct and sufficiently detailed description of your solution and its components. A reader should not have to read your code to understand what you did. Re-using CLIP/Qwen/etc. without showing you understand it costs points.

Results

15 pts

Results presented clearly — input/output examples **and** quantitative evidence (plots, tables, metrics). Most projects should have both.

Presentation

15 pts

Scientific-manuscript quality. Figures from notebooks, unlabeled axes, low-resolution images, or illegible fonts lead to automatic deductions.

Common point deductions — at a glance

Issue	Severity
Including figures borrowed from elsewhere rather than your own	Severe
Screenshots of command line / Jupyter / Colab notebooks used as figures	Severe
Not indicating clearly what is your work vs. not your work (including code)	Severe
No results of your method and no discussion of them	Highest
Figures not exported properly → blurry fonts when zooming	Modest
Text in figures not legible without zooming	Modest
Missing references to the original papers of components you rely on	Modest

On AI assistants & authorship

- Any code you submit is **assumed to be yours** unless indicated otherwise. Code from AI assistants counts as your code.
- No explicit point deduction for using AI assistants per se — but over-reliance typically produces vague technical writing and unoriginal ideas, both of which *do* get deducted under Technical Soundness and Originality.
- Acknowledge AI assistance at the end of the report. Good scholarly practice.

1. Figures Presentation Results

By far the most frequent complaint across the batch of progress reports.

- Fig 1 is the most important figure in your paper.** Look at the figures in the **MovieGen paper** we discussed in class — that is the bar. Not boxes and arrows; a figure that shows real inputs, real outputs, and the structure between them, in one glance.
- Export as vector graphics (PDF or SVG).** Use `savefig("fig.pdf")` from matplotlib, or Illustrator/Inkscape. Pixelated PNGs shrunk into a column are visibly low quality on paper.
- Axis labels, units, legends.** Every axis must say what it is and its units. Every legend entry must be legible at normal reading size. This is non-negotiable.

- **Print your report before submitting.** Many of you had 6-point figure fonts that were readable on a 27" monitor and completely illegible on paper — an automatic Presentation deduction.
- **Design figures for your column width.** If the template is 1-column, design Fig 1 for a single column, or use a 2-column-spanning figure. Do not jam a wide figure into a narrow column.
- **Screenshots of notebooks are not figures.** No Jupyter output, code printouts, or terminal screenshots. Replot cleanly.
- **Each major figure deserves its own paragraph.** Do not lump six unrelated subfigures into "Figure 1." Each experiment gets its own figure and its own discussion.

2. References & related work Technical Originality

- **Use `\cite` with a real bibtex file.** A report without a proper references section does not meet the minimum bar for a technical report. If you mention CLIP, Qwen, LoRA, DINOv2, BLIP, diffusion, or any other component — cite it.
- **Related work must cover *your* problem, not the broad area.** A generic survey of "captioning methods" is not related work for a diagram-understanding project. Find the 5–10 papers that tackle the closest version of your task and engage with them specifically.
- **Do not outsource related work to AI assistants.** They invent citations and miss the papers that matter. Search yourself (Google Scholar, Semantic Scholar, arXiv) and read the abstracts.

3. Technical writing Technical soundness

If I need to read your code to understand what you did, your report has not fulfilled its purpose.

- **Self-contained.** A reasonable computer scientist who has not seen your code should be able to read the report and know what you built, what the inputs/outputs are, and how the pieces connect.
- **Operationalize every vague phrase.** "Structured feedback," "prioritize speed," "clever prompting," "some encoders," "a lightweight model," "prior experience" — replace with what you actually did: which loss, which encoder, what format, which hyperparameters.
- **Define every symbol before using it.** If Eq. (1) uses $\text{sim}(\cdot, \cdot)$, define sim . If you introduce a dataset called "Beans," tell me what it is.

- **Name the exact model.** "Qwen" is not a model — Qwen2.5-VL-7B-Instruct is. "A VLM" is not a model. Be specific.
- **Equations must be correct.** A single wrong-looking equation drags the whole Technical Soundness score down.
- **If you use a pretrained component, show you understand it.** Briefly describe what CLIP / Qwen / SAM does and why it is right for your task. Plugging one in with no commentary reads badly.

4. Results Results

Almost every project needs **both** qualitative and quantitative results.

- **Qualitative.** Real input/output examples from your system. Aim for at least 6 concrete examples — successes *and* failures, side by side. Half a page of examples is usually the right amount. A text-only report will not convince me your system works.
- **Quantitative.** Numbers on a benchmark, with the metric clearly defined, a baseline to compare against, and ideally an ablation. "Our model achieves 81%" means nothing without 81% of what, on what data, versus what baseline.
- **Compare against a sensible baseline.** For almost every project in this class, that is a strong off-the-shelf VLM — Qwen2.5-VL, LLaVA-Next, Gemini-Flash. If an off-the-shelf model already solves your task, your project needs to either improve on it or use it to learn something non-obvious.
- **Evolution matters when relevant.** If you are training something, show metrics through training (e.g. R@K over epochs), not only a final number.
- **Each figure gets a discussion paragraph** — what experiment, what the implications are.

5. Scope & originality Originality

- **Think carefully before training a model from scratch in 2026.** Training a ResNet-18 on 900 samples when Qwen2.5-VL can be prompted or fine-tuned is almost never the right call. If you insist on from-scratch, justify it: what specific insight does that give you that fine-tuning or prompting does not?
- **Prefer fine-tuning or prompting of a foundation model.** SFT, LoRA, or careful prompting almost always gives stronger results within the same compute budget.
- **Dated backbones are a red flag.** If you are using BLIP-1 or similar in 2026, know why. Your insights may apply to modern VLMs but the reader will wonder whether you could have shown

the same on Qwen or Gemma.

- **AI assistants are bad at originality.** Use them for brainstorming and proofreading, not for deciding what your project investigates. If the question came straight out of a ChatGPT prompt, it likely has a generic shape — push past it.

6. Template & structure Presentation

- **Use the template.** Introduction, Related Work, Method, Experiments, Analysis, Conclusion. Reports missing any of these sections were flagged.
- **Put a real title on your report.** "COMP 646 Project" is not a title. The title should describe what you did.
- **Nothing outside the margins.** Tables, figures, code blocks must stay inside the column. Overfull hboxes are a polish deduction.
- **Fix copy-paste residue.** "xxxxxxx", "Lorem ipsum," sentences from a different project — all obvious negative signals.
- **Acknowledge AI assistance** at the end if you used ChatGPT, Gemini, Claude to help write or code.

7. Final self-check before submitting

- ☐ Every figure has labeled axes with units and a readable legend, exported as vector (PDF/SVG)
- ☐ Figure 1 communicates the method in one glance with real inputs/outputs (MovieGen-level)
- ☐ Every external component (CLIP, Qwen, SAM, LoRA, etc.) is cited with `\cite + bibtex`
- ☐ Related work covers neighboring problems specifically, not the whole subfield
- ☐ A reader who has not seen my code can reconstruct what I did from the report alone
- ☐ Every symbol in an equation is defined before use
- ☐ Exact model names (e.g. `Qwen2.5-VL-7B-Instruct`) are used throughout
- ☐ Report includes both qualitative examples (6+, successes and failures) and quantitative numbers
- ☐ At least one strong off-the-shelf baseline is compared against

- ☐ Each major figure has its own paragraph of discussion
- ☐ No content is outside the page margins
- ☐ No placeholder text, no notebook screenshots, no Jupyter output masquerading as figures
- ☐ I have printed the report and checked that everything is readable on paper
- ☐ Report uses the provided template with all required sections, and has a real descriptive title
- ☐ Code link opens and clearly indicates what is mine vs. not mine
- ☐ Any AI assistance is acknowledged at the end
- ☐ The 4-page PDF, code link (and optional 5-min video) are all submitted before 11:59PM May 1st

All the best with your final reports.